

Q-Learning in a Stochastic Gridworld

1. Problem Statement

The objective of this implementation is to demonstrate how a model-free Reinforcement Learning agent (using Q-Learning) adapts its policy when navigating an environment with varying degrees of uncertainty.

In classic deterministic pathfinding algorithms (such as A* or Dijkstra's), an agent assumes perfect execution of its actions. However, real-world environments are inherently stochastic. If a robot attempts to move forward, wheel slip or uneven terrain might cause it to veer off course.

We aim to model this uncertainty using a Markov Decision Process (MDP) in a 5×5 Gridworld. The environment contains a highly rewarding goal state and a series of dangerous “pit” states. By manipulating the “slip probability”—the chance that an agent moves perpendicularly to its intended direction—we aim to observe how the optimal policy transitions from a reckless shortest-path strategy to a cautious, risk-averse strategy.

2. Proposed Solution

To solve this, we propose building a Tabular Q-Learning agent. The solution comprises two main architectural components:

- **The MDP Environment (StochasticGridworld):** A custom grid environment that takes a `slip_probability` parameter. It strictly enforces the rules of the MDP, calculating next states and rewards based on the agent's actions and a random number generator to simulate slipping.
- **The Agent (QLearningAgent):** A model-free learning algorithm that interacts with the environment. It uses an ϵ -greedy strategy to explore the grid and updates a $5 \times 5 \times 4$ Q-table using the Bellman Optimality Equation.

By training two separate agents—one in a deterministic environment ($\text{slip} = 0.0$) and one in a stochastic environment ($\text{slip} = 0.3$)—we can directly compare their converged policies.

3. Code Implementation & Markov Chain Integration

The Python implementation explicitly models the components of a Markov Decision Process (S, A, P, R, γ).

The Environment (State, Actions, Transitions, Rewards)

- **State Space (S):** Represented by `self.current_state = (x, y)`. The grid is 5×5 , creating 25 distinct discrete states.
- **Action Space (A):** `self.actions = [0, 1, 2, 3]` mapping to Up, Right, Down, and Left.
- **Rewards (R):** Handled in the `step()` function. The agent receives +10 for reaching the goal (4,4), -10 for falling into a pit, and a -0.1 step penalty to encourage efficiency.
- **Transition Probabilities (P):** This is the core Markov element of the implementation. In the `step(action)` function, the environment rolls a random variable `roll = np.random.rand()`.
 - If `roll < slip_prob / 2`, the agent slips left of its intended direction.
 - If `roll < slip_prob`, the agent slips right.
 - Otherwise, it moves exactly as intended.

This logic effectively acts as the transition matrix $P(s' | s, a)$ without the agent explicitly knowing the probabilities.

The Q-Learning Agent

The agent initializes a Q-table of zeros: `np.zeros((5, 5, 4))`. It learns purely from the (state, action, reward, next_state) tuples returned by the environment.

The core learning mechanism is the Bellman update:

```
1 td_target = reward + self.gamma * best_next_q
2 td_error = td_target - self.q_table[x, y, action]
3 self.q_table[x, y, action] += self.alpha * td_error
```

Because the transitions are stochastic, the agent falls into the pits multiple times during training. The negative reward propagates backward through the Q-table, mathematically lowering the expected value of taking actions near the pits.

4. Results and Analysis

After training both agents, we extracted the optimal policy `np.argmax(self.q_table[x, y])` for each state. The results clearly demonstrate the agent’s adaptation to stochasticity.

Scenario A: Deterministic Environment (Slip Probability = 0.0)

Output Policy:

```
[ → ] [ → ] [ → ] [ → ] [ G ]
[ → ] [ → ] [ ↑ ] [ ↑ ] [ ↑ ]
[ ↑ ] [ ↑ ] [ ↑ ] [ ↑ ] [ ↑ ]
[ ↑ ] [ X ] [ X ] [ X ] [ ↑ ]
[S↑ ] [ ← ] [ → ] [ → ] [ ↑ ]
```

Analysis: In the deterministic environment, the agent has perfect control. The step penalty (-0.1) drives the agent to find the absolute shortest path to the goal. From the Start state (S), the policy dictates moving Up along the left wall, and then immediately cutting Right. It willingly travels directly adjacent to the deadly pits (X) because the probability of accidentally falling in is 0%. It maximizes reward by minimizing distance.

Scenario B: Stochastic Environment (Slip Probability = 0.3)

Output Policy:

```
[ → ] [ → ] [ → ] [ → ] [ G ]
[ → ] [ → ] [ → ] [ ↑ ] [ ↑ ]
[ ↑ ] [ ↑ ] [ ↑ ] [ ↑ ] [ ↑ ]
[ ← ] [ X ] [ X ] [ X ] [ → ]
[S↑ ] [ ↓ ] [ ↓ ] [ ↓ ] [ ↑ ]
```

Analysis: The introduction of a 30% slip probability fundamentally alters the geometry of the agent’s optimal path.

- **Risk Avoidance:** Notice the tiles immediately below the pits. The policy now dictates moving Down ($\downarrow$) instead of Up or Right. The agent actively pushes itself against the bottom wall. Why? Because if it tries to move Right while adjacent to a pit, a slip could result in an Upward movement, throwing it into the -10 trap.
- **The Wide Detour:** From the Start state, the agent moves Up ($\uparrow$), but instead of cutting Right near the pits, the arrows facing the pits point Left ($\leftarrow$). The agent forces itself against the left wall, traveling all the way to the top-left corner before moving Right across the top row to the Goal.
- **Expected Value:** The agent has mathematically calculated that the cumulative cost of taking extra steps (at -0.1 per step) is far less punishing than the expected value loss of a 15% chance of slipping into a -10 pit.

Conclusion: The experiment successfully demonstrates that Tabular Q-Learning can implicitly learn the stochastic transition dynamics of a Markov environment. Without

being explicitly programmed with the slip probability, the agent discovers and adopts a risk-averse policy purely through experiential learning and the recursive nature of the Bellman equation.